

RAG Generation & Prompt Engineering

3-Layer Prompt · CoT · Faithfulness · Self-Correction · Few-Shot · Citations · Evaluation

WEEK 3 · DAY 18

Thursday, July 23 2026

Day 18 of 84

Sec	Topic	Coverage
Sec 1	Prompt Architecture	3-layer: system (dialect+rules+CoT) + few-shot + user
Sec 2	Chain-of-Thought SQL	5-step CoT: tables→columns→joins→filters→SQL
Sec 3	Faithfulness Enforcement	sqlglot AST parse; detect hallucinated tables/columns
Sec 4	Self-Correction Loop	Generate → validate → re-prompt; max 3 attempts + EXPLAIN
Sec 5	Citation Generation	Model cites tables, columns, joins, filters used
Sec 6	Dynamic Few-Shot	Embed question; retrieve top-k similar examples
Sec 7	Dialect Calibration	PostgreSQL/MySQL/BigQuery/Snowflake syntax blocks
Sec 8	Production Pipeline	Full integration: retrieval (D17) + generation (D18)
Sec 9	Evaluation Framework	Exact match, execution match, faithfulness, latency
Sec 10	FAANG Patterns	Google/Microsoft/Amazon/Meta/Netflix generation
Sec 11	Architect's Lens	Technique ROI order, system prompt rules, SLOs
Sec 12	Exercises + Summary	CoT vs no-CoT, few-shot curve, full pipeline eval

CAPSTONE TEXT-TO-SQL PIPELINE COMPLETE (Days 13-18): Foundation (D13) → vLLM+FastAPI (D14) → Schema RAG (D15) → Indexing Pipeline (D16) → Hybrid Retrieval (D17) → Generation Layer (D18: CoT + faithfulness + self-correction + few-shot). SLOs: syntax valid >98%, faithful >97%, execution accuracy >82%.

Section 1 — Prompt Architecture (3-Layer)

Every SQL generation prompt has three layers. Getting the structure right is the highest-leverage single change you can make.

```
class SQLPromptBuilder:
    """
    Layer 1 – SYSTEM PROMPT: dialect identity + strict rules + CoT protocol
    Layer 2 – FEW-SHOT BLOCK: 2-3 domain-specific question→SQL examples
    Layer 3 – USER MESSAGE: retrieved schema + actual question
    """
    def system_prompt(self) -> str:
        return f"""You are an expert {self.dialect.upper()} SQL engineer.

STRICT RULES:
1. Use ONLY tables and columns listed in the schema. Never invent columns.
2. If required column is missing: SCHEMA_INSUFFICIENT: <reason>
3. Always qualify column names: orders.amount not amount.
4. Write ONE SELECT statement only. No DDL, no DML.
5. {dialect_notes[self.dialect]}
6. End response with: ```sql ... ```

REASONING PROTOCOL (CoT – always follow before writing SQL):
Step 1 – Tables needed to answer the question.
Step 2 – Exact columns needed (quote from schema).
Step 3 – JOIN conditions and FK relationships.
Step 4 – Filters (WHERE), aggregations (GROUP BY), ordering (ORDER BY).
Step 5 – Write the SQL."""

    def few_shot_block(self, examples: list[dict]) -> str:
        """Insert 2-3 domain-specific examples before real question."""
        blocks = []
        for ex in examples[:self.max_examples]:
            blocks.append(
                f"""--- EXAMPLE ---\n
                Schema:\n{ex['schema_context']}\n\n
                Question: {ex['question']}\n\n
                Reasoning:\n{ex['reasoning']}\n\n
                ```sql\n{ex['sql']}\n```")
 return "EXAMPLES:\n\n" + "\n\n".join(blocks)

 def user_message(self, schema_context, question, examples=None) -> str:
 return (
 f"{self.few_shot_block(examples or [])}\n\n"
 f"--- ACTUAL QUESTION ---\n\n"
 f"Schema:\n{schema_context}\n\n"
 f"IMPORTANT: Only use columns that appear verbatim in schema above.\n\n"
 f"Question: {question}")
```

## Section 2 — Chain-of-Thought for SQL Generation

CoT forces schema-grounding before SQL is written. Single highest-impact technique: +10-15% execution accuracy at zero cost.

```
async def generate_sql_cot(question, schema_context, dialect="postgresql") -> dict:
```

```

builder = SQLPromptBuilder(dialect=dialect, use_cot=True, cite_columns=True)
messages = builder.build_messages(schema_context, question)
async with httpx.AsyncClient(base_url=VLLM_URL, timeout=30) as c:
 r = await c.post("/v1/chat/completions", json={
 "model": "sql-llama-8b",
 "messages": messages,
 "max_tokens": 1024,
 "temperature": 0.0, # greedy – SQL is deterministic
 })
 return _parse_cot_response(r.json()["choices"][0]["message"]["content"])

def _parse_cot_response(content: str) -> dict:
 sql_match = re.search(r"```sql\s*(.*?)\s*```", content, re.DOTALL|re.IGNORECASE)
 sql = sql_match.group(1).strip() if sql_match else ""
 reasoning = content[:sql_match.start()].strip() if sql_match else content
 return {"reasoning": reasoning, "sql": sql, "raw": content}

CoT model output for "How many users signed up last month?":
Step 1 – Tables needed: users (has created_at column)
Step 2 – Columns: users.id (COUNT), users.created_at (date filter)
Step 3 – No JOIN needed; single table query
Step 4 – Filter: DATE_TRUNC('month', NOW())-interval '1 month')
```sql
SELECT COUNT(users.id) FROM users
WHERE users.created_at >= DATE_TRUNC('month', NOW()) - INTERVAL '1 month')
AND users.created_at < DATE_TRUNC('month', NOW());
```

```

## Section 3 — Faithfulness Enforcement

Faithfulness: SQL only references tables and columns that exist in the retrieved schema. sqlglot parses the SQL AST to check this in ~5ms before any DB call.

```
import sqlglot, sqlglot.expressions as exp

class SchemaFaithfulnessEnforcer:
 def extract_sql_references(self, sql: str) -> dict:
 """Parse SQL AST – extract all table/column references."""
 try:
 parsed = sqlglot.parse_one(sql, dialect=self.dialect)
 tables = [t.name for t in parsed.find_all(exp.Table)]
 columns = [c.name for c in parsed.find_all(exp.Column)]
 return {"tables": list(set(tables)), "columns": list(set(columns))}
 except sqlglot.errors.ParseError as e:
 return {"error": str(e), "tables": [], "columns": []}

 def check(self, sql: str, schema_context: str) -> dict:
 refs = self.extract_sql_references(sql)
 schema = self.extract_schema_elements(schema_context)
 bad_tables = [t for t in refs["tables"] if t.lower() not in set(schema["tables"])]
 bad_columns = [c for c in refs["columns"] if c.lower() not in set(schema["columns"])
 and c.lower() != "*"]
 return {
 "is_faithful": not bad_tables and not bad_columns,
 "hallucinated_tables": bad_tables,
 "hallucinated_columns": bad_columns,
 }

Example:
SQL: SELECT SUM(o.revenue) FROM orders o JOIN users u ON o.user_id=u.id
Schema: orders(id, amount, user_id), users(id, email)
Result: is_faithful=False, hallucinated_columns=["revenue"] ← schema has "amount"
→ triggers self-correction with error message
```

## Section 4 — Self-Correction Loop

```
class SelfCorrectingGenerator:
 MAX_ATTEMPTS = 3

 async def generate_with_correction(self, question, schema_context, examples=None):
 messages = self.builder.build_messages(schema_context, question, examples)
 last_error = None

 for attempt in range(1, self.MAX_ATTEMPTS + 1):
 # Append correction if previous attempt failed
 if last_error:
 messages.append({"role": "assistant", "content": last_result["raw"]})
 messages.append({"role": "user", "content":
 f"Your SQL has an error. Fix it.\n\n"
 f"ERROR: {last_error}\n\n"
 f"Schema (reference):\n{schema_context}\n\nCorrected SQL:"})

 result = await generate_sql_cot(question, schema_context)
```

```

last_result = result
if not result["sql"]:
 last_error = "No SQL block found in response."; continue

1. Faithfulness check (fast, ~5ms, no DB)
faith = self.enforcer.check(result["sql"], schema_context)
if not faith["is_faithful"]:
 last_error = (f"Hallucinated: tables={faith['hallucinated_tables']}, "
 f"cols={faith['hallucinated_columns']}")
 FAITHFUL_FAIL.inc(); continue

2. Dry-run EXPLAIN (validate syntax, ~10ms DB call)
exec_err = await self._dry_run(result["sql"])
if exec_err:
 last_error = f"Execution error: {exec_err}"; continue

return {**result, "attempts": attempt, "success": True}

return {"sql":None,"error":last_error,"attempts":self.MAX_ATTEMPTS,"success":False}

async def _dry_run(self, sql: str) -> str | None:
 """EXPLAIN validates syntax without touching data."""
 try:
 conn = await asyncpg.connect(self.db_url)
 await conn.execute(f"EXPLAIN {sql}")
 await conn.close(); return None
 except Exception as e: return str(e)

Correction example:
Attempt 1: SUM(o.revenue) → hallucinated_columns: ["revenue"] → error injected
Attempt 2: SUM(o.amount) → faithful=True, EXPLAIN=OK → success at attempt 2

```

## Section 5 — Citation Generation

```
CITATION_SYSTEM = """After generating SQL, provide a citations block:

\```citations
TABLES_USED: <comma-separated>
COLUMNS_USED: <table.column pairs>
JOIN_CONDITIONS: <table1.col = table2.col>
FILTERS_APPLIED: <plain-English description>
ANSWER_BASIS: <one sentence: what the SQL measures>
\```"""

def parse_citations(raw_response: str) -> dict:
 match = re.search(r"\```citations\s*(.*?)\s*\```", raw_response, re.DOTALL|re.IGNORECASE)
 if not match: return {}
 result = {}
 for line in match.group(1).splitlines():
 if ":" in line:
 k,v = line.split(":",1)
 result[k.strip().lower().replace(" ","_")] = v.strip()
 return result

Output for "How many users ordered in the last 30 days?":
{
"tables_used": "users, orders",
"columns_used": "users.id, orders.user_id, orders.created_at",
"join_conditions": "users.id = orders.user_id",
"filters_applied": "orders.created_at >= NOW() - INTERVAL '30 days'",
"answer_basis": "Counts distinct users with at least one order in last 30 days."
}
```

## Section 6 — Dynamic Few-Shot Selection

Embed the question, retrieve the top-k most similar examples from the bank. Dynamic selection consistently adds 5-10pp over static examples.

```
class DynamicFewShotSelector:
 """
 Pre-embed all example questions at startup (O(n) once).
 At query time: dot product similarity → top-k (O(n) per query, sub-ms for n<1000).
 """
 def __init__(self, example_bank, embedder_name="BAAI/bge-large-en-v1.5", top_k=3):
 self.bank = example_bank
 self.top_k = top_k
 self.model = SentenceTransformer(embedder_name)
 questions = [ex["question"] for ex in example_bank]
 self.q_vecs = self.model.encode(questions, normalize_embeddings=True)

 def select(self, question: str) -> list[dict]:
 q_vec = self.model.encode(question, normalize_embeddings=True)
 scores = np.dot(self.q_vecs, q_vec) # cosine sim (vecs normalised)
 top_k = np.argsort(scores)[::-1][:self.top_k]
 return [self.bank[i] for i in top_k]

Critical example patterns in the bank:
```

```
1. Anti-join (LEFT JOIN + IS NULL) → handles "users who never ordered"
2. GROUP BY + HAVING → handles "products with more than 100 orders"
3. Window function (ROW_NUMBER) → handles "latest order per user"
4. Subquery / CTE → handles "users above average spend"
5. Date truncation → handles "orders this month / last quarter"

Rule: 20-50 examples in bank; select top-3 dynamically
Finding: 3-5 examples captures 80% of the accuracy gain vs 20+ examples
```

## Section 7 — Dialect Calibration

```
DIALECT_EXAMPLES = {
 "postgresql": {
 "date_trunc": "DATE_TRUNC('month', created_at)",
 "string_match": "email ILIKE '%@gmail.com'",
 "json_field": "metadata->'key'",
 "window": "ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY created_at DESC)",
 },
 "mysql": {
 "date_trunc": "DATE_FORMAT(created_at, '%Y-%m-01')",
 "string_match": "email LIKE '%@gmail.com'",
 "json_field": "JSON_EXTRACT(metadata, '$.key')",
 },
 "bigquery": {
 "date_trunc": "TIMESTAMP_TRUNC(created_at, MONTH)",
 "string_match": "LOWER(email) LIKE '%@gmail.com'",
 "json_field": "JSON_VALUE(metadata, '$.key')",
 },
 "snowflake": {
 "date_trunc": "DATE_TRUNC('MONTH', created_at)",
 "string_match": "email ILIKE '%@gmail.com'",
 "json_field": "metadata:key::STRING",
 },
}

Injected into system prompt as a calibration block:
"DIALECT: POSTGRESQL. Use these exact syntax patterns:
date_trunc: DATE_TRUNC('month', created_at)
string_match: email ILIKE '%@gmail.com'
..."

Common dialect mismatch failures (production gotchas):
MySQL: no ILIKE → use LOWER(col) LIKE '%pattern%'
SQLite: no window functions before 3.25; no RIGHT JOIN
BigQuery: backtick project.dataset.table identifiers
Snowflake: QUALIFY clause instead of subquery for window filter
```

## Section 8 — Production Pipeline (Full Integration)

GENERATION LAYER COMPLETE. Full Text-to-SQL pipeline: Day 17 HybridSchemaRetriever → Day 18 ProductionSQLGenerator. SLOs: syntax valid >98%, faithful >97%, execution accuracy >82%, latency <3s.

```
class ProductionSQLGenerator:
 """
 Full pipeline:
 1. Dynamic few-shot selection (embedding similarity)
 2. CoT generation (5-step reasoning before SQL)
 3. Faithfulness enforcement (sqlglot AST)
 4. Self-correction (up to 3 attempts, with EXPLAIN dry-run)
 5. Citation extraction
 6. Prometheus instrumentation
```



```

"""
async def generate(self, question: str, schema_context: str,
 tenant_id: str) -> dict:
 t0 = time.perf_counter()
 examples = self.shot_selector.select(question)
 result = await self.generator.generate_with_correction(
 question, schema_context, examples)
 latency = time.perf_counter() - t0
 GEN_LATENCY.labels(dialect=self.dialect,
 attempt=result.get("attempts",1)).observe(latency)
 if result["success"]:
 GEN_ATTEMPTS.labels(outcome="success").inc()
 return {
 "sql": result["sql"],
 "reasoning": result["reasoning"],
 "citations": parse_citations(result.get("raw", "")),
 "attempts": result["attempts"],
 "latency_ms": round(latency*1000),
 "success": True,
 }
 GEN_ATTEMPTS.labels(outcome="failure").inc()
 return {"sql":None,"error":result.get("error"),"success":False}

FastAPI integration (wires into Day 14 /sql endpoint):
POST /sql
→ HybridSchemaRetriever.retrieve() [Day 17] → top-5 tables
→ ProductionSQLGenerator.generate() [Day 18] → SQL + citations
→ Return: {sql, reasoning, citations, attempts, latency_ms}

```

## Section 9 — Evaluation Framework

```

class GenerationEvaluator:
 async def evaluate(self, cases: list[GenerationEvalCase]) -> GenerationEvalResult:
 res = GenerationEvalResult(total=len(cases))
 conn = await asyncpg.connect(self.db_url)
 for case in cases:
 gen = await self.generator.generate(case.question, case.schema_context, "eval")
 res.avg_latency_ms += gen.get("latency_ms", 0)
 res.avg_attempts += gen.get("attempts", 1)
 if not gen["success"] or not gen["sql"]: continue

 # Syntax valid (sqlglot parse)
 if "error" not in self.enforcer.extract_sql_references(gen["sql"]):
 res.syntax_valid += 1
 # Faithfulness (AST vs schema)
 if self.enforcer.check(gen["sql"], case.schema_context)["is_faithful"]:
 res.faithful += 1
 # Exact match (normalise whitespace)
 if " ".join(gen["sql"].lower().split()) == " ".join(case.gold_sql.lower().split()):
 res.exact_match += 1
 # Execution match (run both queries, compare result sets)
 try:
 gen_rows = [dict(r) for r in await conn.fetch(gen["sql"])]
 gold_rows = [dict(r) for r in await conn.fetch(case.gold_sql)]
 if sorted(str(r) for r in gen_rows) == sorted(str(r) for r in gold_rows):
 res.execution_match += 1
 except Exception: pass
 await conn.close()
 n = len(cases)
 res.avg_latency_ms /= n; res.avg_attempts /= n
 return res

PRODUCTION TARGETS:
syntax_valid / total > 0.98 (nearly all SQL is parseable)
faithful / total > 0.97 (almost no hallucinated columns)
execution_match/total > 0.82 (BIRD benchmark SOTA ~85%)
avg_attempts < 1.3 (most queries succeed first try)
avg_latency_ms < 3000 (including up to 1 correction)

```

## Section 10 — FAANG Generation Patterns

### Google — Grammar-Constrained Decoding (Google Sheets NL to Formula)

Token distribution masked at each decoding step to allow only tokens forming a valid spreadsheet formula per a context-free grammar (CFG). Result: 0% syntax error rate. Trade-off: grammar constraint evaluation adds 15-30% generation time. Syntax errors are structurally impossible — no correction loop needed.

### Microsoft — LLMingua Prompt Compression (Copilot for SQL)

Databases with 200+ tables exceed context windows. LLMingua (prompt compression model) removes low-information tokens from schema context: 40K tokens → 8K tokens, under 3% information loss (measured by execution accuracy delta). Runs as CPU pre-processing step in under 200ms before the LLM generation call.

### Amazon — Execution-Guided Beam Search (QuickSight Q)

Generate beam of 5 SQL candidates simultaneously. Execute all 5 against DB in parallel. Select the candidate that: (1) executes without error, (2) returns non-empty result, (3) has the most focused result set (fewest rows still answering the question). Result: 89% execution accuracy vs 76% greedy decoding on Amazon's internal BI benchmark.

### Meta — Two-Stage Schema Linking + SQL Synthesis (Internal Data Access)

Stage 1: small fine-tuned model (LLaMA 3 8B) predicts relevant tables/columns (schema linking). Stage 2: larger model (LLaMA 3 70B) generates SQL from the filtered mini-schema (85% smaller than full schema). Stage 1 runs fast; Stage 2 sees less noise. End-to-end accuracy: +11pp over single-stage on Meta's internal benchmark.

### Netflix — Prompt Versioning and A/B Testing (Content Analytics)

Every prompt template version tracked in a configuration service. New templates A/B tested: 10% traffic to new template, 90% to current. Promote only if statistically significant improvement ( $p < 0.05$ ) with no regression on passing tests. One change — 'Always qualify column names with their table name' — increased execution accuracy from 81% to 87% in one deployment, discovered via this A/B framework.

## Section 11 — Architect's Lens

TECHNIQUE ROI ORDER (add in this sequence for maximum accuracy gain):

- |                                       |                                        |
|---------------------------------------|----------------------------------------|
| 1. CoT in system prompt               | → +10-15% exec accuracy; FREE          |
| 2. Column qualification rule          | → +5-8%; FREE                          |
| 3. Dialect-specific system prompt     | → eliminates dialect errors; FREE      |
| 4. Dynamic few-shot (top-k examples)  | → +5-10%; embedding cost at query time |
| 5. Self-correction loop (max 3)       | → +8-12%; 2-3x latency cost            |
| 6. Faithfulness enforcement (sqlglot) | → hallucination detection; ~5ms        |
| 7. Dry-run EXPLAIN validation         | → syntax validation; ~10ms DB call     |
| 8. Citation generation                | → debuggability; no accuracy gain      |
| 9. Grammar-constrained decoding       | → 0% syntax errors; +15-30% latency    |

SYSTEM PROMPT RULES (ranked by accuracy impact):

- |                                                      |                                   |
|------------------------------------------------------|-----------------------------------|
| 1. "Use ONLY tables/columns in the provided schema"  | → biggest single impact           |
| 2. "Qualify all column names: table.column"          | → eliminates ambiguity errors     |
| 3. "If schema insufficient, say SCHEMA_INSUFFICIENT" | → prevents hallucination          |
| 4. "Write ONE SELECT statement only"                 | → prevents multi-statement output |
| 5. "Use {dialect} syntax specifically"               | → eliminates dialect errors       |

FEW-SHOT EXAMPLE BANK — MUST INCLUDE:

- |                                           |                               |
|-------------------------------------------|-------------------------------|
| - Anti-join pattern (LEFT JOIN + IS NULL) | → "never ordered" queries     |
| - GROUP BY + HAVING                       | → "more than X" queries       |
| - Window function (ROW_NUMBER/RANK)       | → "latest per user" queries   |
| - Subquery or CTE                         | → rank/percentile queries     |
| - Date truncation + interval filter       | → "this month / last quarter" |

Bank size: 20-50 examples; select top-3 dynamically per question

GENERATION SLO TARGETS:

Syntax valid rate	: > 98%
Faithfulness rate	: > 97%
Execution accuracy	: > 82% (BIRD benchmark SOTA ~85%)
Avg generation latency:	< 3s (including 1 correction attempt)
Avg attempts	: < 1.3 (most queries succeed first try)

## Section 12 — Exercises and Summary

### Exercise 1: CoT vs No-CoT Accuracy

```
10 questions: Easy (single table), Medium (2-table JOIN), Hard (subquery/window)
Run each with use_cot=True and use_cot=False
Measure execution accuracy per difficulty tier
Expected: CoT advantage grows with question complexity
Typical finding:
Easy: CoT=0.92 vs No-CoT=0.88 (+4pp)
Medium: CoT=0.85 vs No-CoT=0.73 (+12pp)
Hard: CoT=0.72 vs No-CoT=0.51 (+21pp)
```

### Exercise 2: Few-Shot Example Curve

```
Measure execution accuracy at: 0, 1, 3, 5, 10 examples
Plot: examples_count → execution_accuracy
Find the knee of the curve (typically at 3-5 examples)
0 examples: ~0.68 baseline
1 example: ~0.74 (+6pp)
3 examples: ~0.80 (+6pp more)
5 examples: ~0.82 (+2pp more – diminishing returns)
10 examples: ~0.83 (+1pp more – nearly saturated)
```

### Summary

Concept	Key Point
3-layer prompt	System (dialect+rules+CoT) + few-shot + user (schema+question)
Chain-of-thought	5 steps: tables→columns→joins→filters→SQL; +10-15% accuracy
Faithfulness	sqlglot AST parse; detect hallucinated tables/columns; ~5ms
Self-correction	Generate→validate→re-prompt with error; max 3 attempts
Citation generation	Model cites tables, columns, joins used; essential for debug
Dynamic few-shot	Embed question; retrieve top-k similar examples; +5-10%
Dialect calibration	Per-dialect syntax block in system prompt; eliminates errors
Execution accuracy	> 82% target; BIRD SOTA ~85%; measure via result set match
Production SLOs	Syntax valid >98%, faithful >97%, latency <3s, attempts <1.3
Capstone complete	Days 13-18: full Text-to-SQL production pipeline built

### NEXT: Day 19 — Agents and Tool Use

Building agents that call tools: ReAct pattern, tool schemas, multi-step reasoning loops, agent memory, error handling, and the orchestration layer for autonomous Text-to-SQL agents.